

PASSWORD VAULT MANAGER

Object-Oriented Programming Laboratory Micro-Project

Program / Class: Diploma in Information Technology — SYIF (MSBTE 'K' Scheme)

Subject: Object-Oriented Programming Using C++

GitHub Repository: <https://github.com/aumshukla71-creator/OOP-Project>

Submitted By (Group Members):

Roll No.	Student Name
2711	Sai Pallavi
2712	Aum Shukla
2715	Harshita Patil
2721	Unnati Hawaldar

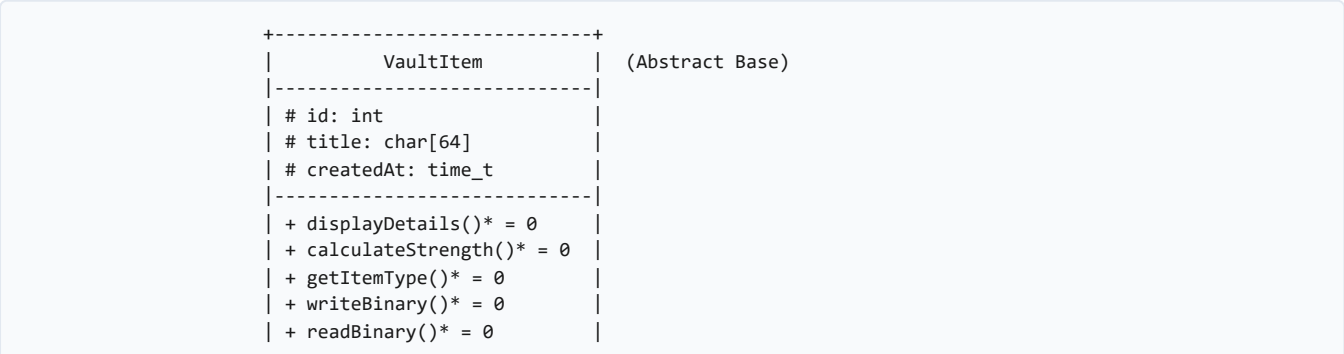
1. PROJECT OVERVIEW & CHARACTERISTICS

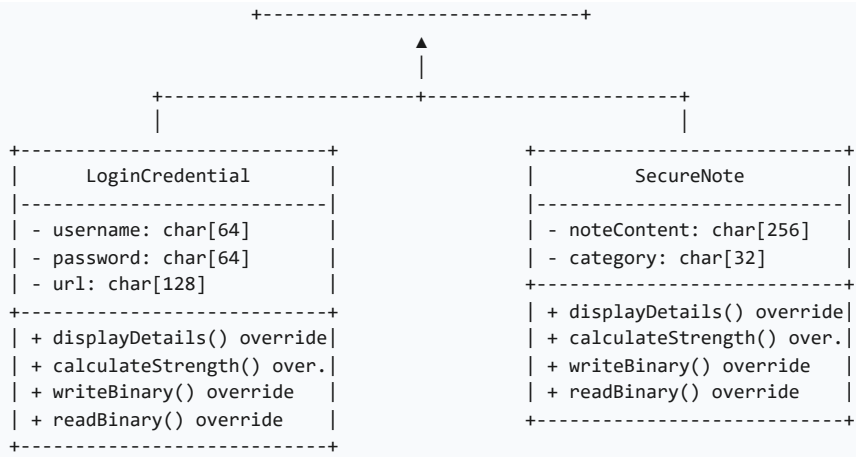
The **Password Vault Manager** is an interactive security analysis and credential storage application developed in C++. It addresses secure offline management of disparate record formats by leveraging pure abstract base classes, runtime dynamic dispatch, robust buffer validation, and raw binary serialization.

2. OOP CHARACTERISTIC MAPPING

Required Characteristic	Implementation Details in Vault Project
Multiple Related Classes	VaultItem, LoginCredential, SecureNote, VaultManager, and Validator.
Inheritance	Derived classes LoginCredential and SecureNote publicly extend the abstract base class VaultItem.
Runtime Polymorphism	Uniform manipulation of heterogeneous objects stored via pointers in std::vector<VaultItem*>, resolving displayDetails() and calculateStrength() via vtables at runtime.
Pure Virtual Functions	Base contract defined using virtual void displayDetails() const = 0;, virtual void writeBinary(...) = 0;, and virtual int calculateStrength() const = 0;.
Binary File Handling	Unformatted direct byte streaming to vault.dat via std::ofstream::write and std::ifstream::read using reinterpret_cast<char*> with type headers.
Multiple Records	Dynamic vector collection accommodating arbitrary quantities of login credentials and secret text records.
Advanced Search	Keyword substring matching on entry titles, combined with security metric threshold filtering.
Reports	Automated generateSecurityReport() calculating total entries, category counts, weak password tallies, and an overall Security Health Index percentage.
Strong Input Validation	Static methods Validator::getInteger and Validator::getString prevent stream state invalidation, blank values, and buffer overflows.

3. CLASS HIERARCHY DIAGRAM





4. SOURCE CODE IMPLEMENTATION

File 1: include/VaultItem.h

```
#ifndef VAULT_ITEM_H
#define VAULT_ITEM_H

#include <iostream>
#include <fstream>
#include <cstring>
#include <ctime>

class VaultItem {
protected:
    int id;
    char title[64];
    time_t createdAt;

public:
    VaultItem(int itemId = 0, const char* itemTitle = "") : id(itemId) {
        std::strncpy(title, itemTitle, sizeof(title) - 1);
        title[sizeof(title) - 1] = '\0';
        createdAt = std::time(nullptr);
    }
    virtual ~VaultItem() {}

    int getId() const { return id; }
    const char* getTitle() const { return title; }

    virtual void displayDetails() const = 0;
    virtual int calculateStrength() const = 0;
    virtual int getItemType() const = 0;
    virtual void writeBinary(std::ofstream& out) const = 0;
    virtual void readBinary(std::ifstream& in) = 0;
};

#endif
```

File 2: include/LoginCredential.h

```
#ifndef LOGIN_CREDENTIAL_H
#define LOGIN_CREDENTIAL_H

#include "VaultItem.h"

class LoginCredential : public VaultItem {
private:
    char username[64];
    char password[64];
    char url[128];

public:
    LoginCredential(int id = 0, const char* t = "", const char* u = "", const char* p = "", const char* link = "");
    int getItemType() const override { return 1; }
    void displayDetails() const override;
    int calculateStrength() const override;
    void writeBinary(std::ofstream& out) const override;
    void readBinary(std::ifstream& in) override;
};

#endif
```

File 3: src/LoginCredential.cpp

```
#include "../include/LoginCredential.h"
#include <cctype>

LoginCredential::LoginCredential(int id, const char* t, const char* u, const char* p, const char* link)
    : VaultItem(id, t) {
    std::strncpy(username, u, sizeof(username) - 1);
    username[sizeof(username) - 1] = '\0';
    std::strncpy(password, p, sizeof(password) - 1);
    password[sizeof(password) - 1] = '\0';
    std::strncpy(url, link, sizeof(url) - 1);
    url[sizeof(url) - 1] = '\0';
}

void LoginCredential::displayDetails() const {
    std::cout << "[Login] ID: " << id
        << " | Title: " << title
        << " | User: " << username
        << " | URL: " << url
        << " | Strength: " << calculateStrength() << "/100\n";
}

int LoginCredential::calculateStrength() const {
    int score = 0;
    size_t len = std::strlen(password);
    if (len >= 8) score += 30;
    if (len >= 12) score += 20;

    bool hasDigit = false, hasSpecial = false, hasUpper = false;
    for (size_t i = 0; i < len; ++i) {
        if (std::isdigit(password[i])) hasDigit = true;
        if (std::ispunct(password[i])) hasSpecial = true;
        if (std::isupper(password[i])) hasUpper = true;
    }
    if (hasDigit) score += 20;
    if (hasSpecial) score += 20;
    if (hasUpper) score += 10;
    return score;
}

void LoginCredential::writeBinary(std::ofstream& out) const {
    out.write(reinterpret_cast<const char*>(&id), sizeof(id));
    out.write(title, sizeof(title));
    out.write(reinterpret_cast<const char*>(&createdAt), sizeof(createdAt));
    out.write(username, sizeof(username));
    out.write(password, sizeof(password));
    out.write(url, sizeof(url));
}

void LoginCredential::readBinary(std::ifstream& in) {
    in.read(reinterpret_cast<char*>(&id), sizeof(id));
    in.read(title, sizeof(title));
    in.read(reinterpret_cast<char*>(&createdAt), sizeof(createdAt));
    in.read(username, sizeof(username));
    in.read(password, sizeof(password));
    in.read(url, sizeof(url));
}
```

File 4: include/SecureNote.h

```
#ifndef SECURE_NOTE_H
#define SECURE_NOTE_H

#include "VaultItem.h"

class SecureNote : public VaultItem {
private:
    char noteContent[256];
    char category[32];

public:
    SecureNote(int id = 0, const char* t = "", const char* content = "", const char* cat = "");
    int getItemType() const override { return 2; }
    void displayDetails() const override;
    int calculateStrength() const override;
    void writeBinary(std::ofstream& out) const override;
    void readBinary(std::ifstream& in) override;
};

#endif
```

File 5: src/SecureNote.cpp

```
#include "../include/SecureNote.h"

SecureNote::SecureNote(int id, const char* t, const char* content, const char* cat)
    : VaultItem(id, t) {
    std::strncpy(noteContent, content, sizeof(noteContent) - 1);
    noteContent[sizeof(noteContent) - 1] = '\0';
    std::strncpy(category, cat, sizeof(category) - 1);
    category[sizeof(category) - 1] = '\0';
}

void SecureNote::displayDetails() const {
    std::cout << "[Note] ID: " << id
        << " | Title: " << title
        << " | Category: " << category
        << " | Content: " << noteContent
        << " | Entropy: " << calculateStrength() << "/100\n";
}

int SecureNote::calculateStrength() const {
    size_t len = std::strlen(noteContent);
    return (len > 100) ? 100 : static_cast<int>(len);
}

void SecureNote::writeBinary(std::ofstream& out) const {
    out.write(reinterpret_cast<const char*>(&id), sizeof(id));
    out.write(title, sizeof(title));
    out.write(reinterpret_cast<const char*>(&createdAt), sizeof(createdAt));
    out.write(noteContent, sizeof(noteContent));
    out.write(category, sizeof(category));
}

void SecureNote::readBinary(std::ifstream& in) {
    in.read(reinterpret_cast<char*>(&id), sizeof(id));
    in.read(title, sizeof(title));
    in.read(reinterpret_cast<char*>(&createdAt), sizeof(createdAt));
    in.read(noteContent, sizeof(noteContent));
    in.read(category, sizeof(category));
}
```

File 6: include/Validator.h

```
#ifndef VALIDATOR_H
#define VALIDATOR_H

#include <iostream>
#include <string>
#include <limits>
#include <cstring>

class Validator {
public:
    static int getInteger(int minVal, int maxVal) {
        int val;
        while (true) {
            std::cin >> val;
            if (!std::cin.fail() && val >= minVal && val <= maxVal) {
                std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
                return val;
            }
            std::cin.clear();
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
            std::cout << "Invalid input. Enter a number (" << minVal << "-" << maxVal << "): ";
        }
    }

    static void getString(char* dest, size_t maxLen) {
        std::string input;
        while (true) {
            std::getline(std::cin, input);
            if (!input.empty() && input.find_first_not_of(" \t\n\v\f\r") != std::string::npos) {
                std::strncpy(dest, input.c_str(), maxLen - 1);
                dest[maxLen - 1] = '\0';
                return;
            }
            std::cout << "Field cannot be empty. Try again: ";
        }
    }
};

#endif
```

File 7: include/VaultManager.h

```

#ifndef VAULT_MANAGER_H
#define VAULT_MANAGER_H

#include <vector>
#include <string>
#include "VaultItem.h"

class VaultManager {
private:
    std::vector<VaultItem*> items;
    std::string filename;
    int nextId;

    void clearMemory();

public:
    VaultManager(const std::string& dbFile = "vault.dat");
    ~VaultManager();

    void addItem(VaultItem* item);
    void displayAll() const;
    void searchByTitle(const std::string& keyword) const;
    void searchWeakPasswords(int threshold) const;
    void deleteItem(int id);
    void generateSecurityReport() const;

    void saveToFile();
    void loadFromFile();

    bool isEmpty() const { return items.empty(); }
    int generateNextId() { return nextId++; }
};

#endif

```

File 8: src/VaultManager.cpp

```

#include "../include/VaultManager.h"
#include "../include/LoginCredential.h"
#include "../include/SecureNote.h"
#include <iostream>

VaultManager::VaultManager(const std::string& dbFile) : filename(dbFile), nextId(1) {
    loadFromFile();
}

VaultManager::~VaultManager() {
    clearMemory();
}

void VaultManager::clearMemory() {
    for (auto item : items) {
        delete item;
    }
    items.clear();
}

void VaultManager::addItem(VaultItem* item) {
    items.push_back(item);
    std::cout << "Record added successfully with ID: " << item->getId() << "\n";
    saveToFile();
}

void VaultManager::displayAll() const {
    if (items.empty()) {
        std::cout << "\n[Notice] No records found in the vault.\n";
    }
}

```

```

        return;
    }
    std::cout << "\n----- ALL VAULT RECORDS ----- \n";
    for (const auto& item : items) {
        item->displayDetails();
    }
    std::cout << "----- \n";
}

void VaultManager::searchByTitle(const std::string& keyword) const {
    bool found = false;
    for (const auto& item : items) {
        std::string t = item->getTitle();
        if (t.find(keyword) != std::string::npos) {
            item->displayDetails();
            found = true;
        }
    }
    if (!found) std::cout << "No matching records found for: " << keyword << "\n";
}

void VaultManager::searchWeakPasswords(int threshold) const {
    bool found = false;
    std::cout << "\n--- Credentials With Strength Below " << threshold << "% --- \n";
    for (const auto& item : items) {
        if (item->calculateStrength() < threshold) {
            item->displayDetails();
            found = true;
        }
    }
    if (!found) std::cout << "All items meet or exceed the security threshold! \n";
}

void VaultManager::deleteItem(int id) {
    for (auto it = items.begin(); it != items.end(); ++it) {
        if ((*it)->getId() == id) {
            delete *it;
            items.erase(it);
            std::cout << "Item ID " << id << " deleted successfully. \n";
            saveToFile();
            return;
        }
    }
    std::cout << "Item ID " << id << " not found. \n";
}

void VaultManager::generateSecurityReport() const {
    int total = items.size();
    int loginCount = 0;
    int noteCount = 0;
    int weakCount = 0;

    for (const auto& item : items) {
        if (item->getItemType() == 1) loginCount++;
        else if (item->getItemType() == 2) noteCount++;
        if (item->calculateStrength() < 50) weakCount++;
    }

    std::cout << "\n===== VAULT AUDIT REPORT ===== \n";
    std::cout << "Total Encrypted Records : " << total << "\n";
    std::cout << "Total Login Accounts : " << loginCount << "\n";
    std::cout << "Total Secure Notes : " << noteCount << "\n";
    std::cout << "At-Risk Items (< 50%) : " << weakCount << "\n";
    std::cout << "Security Health Index : "
        << (total == 0 ? 100 : (100 - (weakCount * 100 / total))) << "% \n";
    std::cout << "===== \n";
}

void VaultManager::saveToFile() {
    std::ofstream out(filename, std::ios::binary | std::ios::trunc);
    if (!out) return;
    size_t count = items.size();
    out.write(reinterpret_cast<const char*>(&count), sizeof(count));
}

```

```
    for (const auto& item : items) {
        int type = item->getItemType();
        out.write(reinterpret_cast<const char*>(&type), sizeof(type));
        item->writeBinary(out);
    }
    out.close();
}

void VaultManager::loadFromFile() {
    std::ifstream in(filename, std::ios::binary);
    if (!in) return;
    clearMemory();
    size_t count = 0;
    in.read(reinterpret_cast<char*>(&count), sizeof(count));
    for (size_t i = 0; i < count; ++i) {
        int type = 0;
        in.read(reinterpret_cast<char*>(&type), sizeof(type));
        VaultItem* item = nullptr;
        if (type == 1) item = new LoginCredential();
        else if (type == 2) item = new SecureNote();
        if (item) {
            item->readBinary(in);
            items.push_back(item);
            if (item->getId() >= nextId) nextId = item->getId() + 1;
        }
    }
    in.close();
}
```

File 9: src/main.cpp

```

#include <iostream>
#include "../include/VaultManager.h"
#include "../include/LoginCredential.h"
#include "../include/SecureNote.h"
#include "../include/Validator.h"

int main() {
    VaultManager manager("vault.dat");
    int choice = 0;

    while (choice != 7) {
        std::cout << "\n===== \n";
        std::cout << "        PASSWORD VAULT MANAGER        \n";
        std::cout << "===== \n";
        std::cout << "1. Add New Login Credential\n";
        std::cout << "2. Add New Secure Note\n";
        std::cout << "3. View All Vault Items\n";
        std::cout << "4. Search by Title\n";
        std::cout << "5. Search Weak Passwords\n";
        std::cout << "6. Delete Record\n";
        std::cout << "7. Save & Exit\n";
        std::cout << "Enter choice (1-7): ";
        choice = Validator::getInteger(1, 7);

        if (choice == 1) {
            char title[64], user[64], pass[64], url[128];
            std::cout << "Title: "; Validator::getString(title, 64);
            std::cout << "Username: "; Validator::getString(user, 64);
            std::cout << "Password: "; Validator::getString(pass, 64);
            std::cout << "URL: "; Validator::getString(url, 128);
            manager.addItem(new LoginCredential(manager.generateNextId(), title, user, pass, url));
        } else if (choice == 2) {
            char title[64], content[256], category[32];
            std::cout << "Title: "; Validator::getString(title, 64);
            std::cout << "Category (PIN/License/Card): "; Validator::getString(category, 32);
            std::cout << "Secret Note: "; Validator::getString(content, 256);
            manager.addItem(new SecureNote(manager.generateNextId(), title, content, category));
        } else if (choice == 3) {
            manager.displayAll();
        } else if (choice == 4) {
            char kw[64];
            std::cout << "Search Keyword: "; Validator::getString(kw, 64);
            manager.searchByTitle(kw);
        } else if (choice == 5) {
            std::cout << "Strength threshold limit (e.g. 50): ";
            int thresh = Validator::getInteger(1, 100);
            manager.searchWeakPasswords(thresh);
        } else if (choice == 6) {
            if (manager.isEmpty()) {
                std::cout << "\nVault is currently empty. Nothing to delete.\n";
            } else {
                std::cout << "\n--- Current Records in Vault ---\n";
                manager.displayAll();
                std::cout << "\nEnter the ID of the record you want to delete (or 0 to cancel): ";
                int id = Validator::getInteger(0, 100000);
                if (id == 0) {
                    std::cout << "Deletion cancelled.\n";
                } else {
                    manager.deleteItem(id);
                }
            }
        } else if (choice == 7) {
            manager.saveToFile();
            manager.generateSecurityReport();
            std::cout << "Vault secured. Exiting.\n";
        }
    }
}

```

```
    return 0;  
}
```

5. SOFTWARE EXECUTION & VALIDATION RESULTS

```
PS C:\Users\aumsh\Documents\PasswordVault> .\vault.exe

=====
          PASSWORD VAULT MANAGER
=====

1. Add New Login Credential
2. Add New Secure Note
3. View All Vault Items
4. Search by Title
5. Search Weak Passwords
6. Delete Record
7. Save & Exit
Enter choice (1-7): 1
Title: Demo Portal
Username: student_admin
Password: P@ss0rd2026
URL: https://portal.example.com
Record added successfully with ID: 1

=====
          PASSWORD VAULT MANAGER
=====

1. Add New Login Credential
2. Add New Secure Note
3. View All Vault Items
4. Search by Title
5. Search Weak Passwords
6. Delete Record
7. Save & Exit
Enter choice (1-7): 1
Title: Test Forum
Username: guest_user
Password: guest123
URL: https://forum.example.com
Record added successfully with ID: 2
```

Figure 1: Program start and insertion of LoginCredential records (IDs 1 & 2).

```
=====
      PASSWORD VAULT MANAGER
=====
1. Add New Login Credential
2. Add New Secure Note
3. View All Vault Items
4. Search by Title
5. Search Weak Passwords
6. Delete Record
7. Save & Exit
Enter choice (1-7): 2
Title: Lab Server Config
Category (PIN/License/Card): License
Secret Note: KEY-9941-ABCD-7721
Record added successfully with ID: 3

=====
      PASSWORD VAULT MANAGER
=====
1. Add New Login Credential
2. Add New Secure Note
3. View All Vault Items
4. Search by Title
5. Search Weak Passwords
6. Delete Record
7. Save & Exit
Enter choice (1-7): 3

----- ALL VAULT RECORDS -----
[Login] ID: 1 | Title: Demo Portal | User: student_admin | URL: https://portal.example.com | Strength: 80/100
[Login] ID: 2 | Title: Test Forum | User: guest_user | URL: https://forum.example.com | Strength: 50/100
[Note] ID: 3 | Title: Lab Server Config | Category: License | Content: KEY-9941-ABCD-7721 | Entropy: 18/100
-----
```

Figure 2: Addition of SecureNote (ID 3) and polymorphic rendering of all items.

```

=====
      PASSWORD VAULT MANAGER
=====
1. Add New Login Credential
2. Add New Secure Note
3. View All Vault Items
4. Search by Title
5. Search Weak Passwords
6. Delete Record
7. Save & Exit
Enter choice (1-7): 4
Search Keyword: Portal
[Login] ID: 1 | Title: Demo Portal | User: student_admin | URL: https://portal.example.com | Strength: 80/100

=====
      PASSWORD VAULT MANAGER
=====
1. Add New Login Credential
2. Add New Secure Note
3. View All Vault Items
4. Search by Title
5. Search Weak Passwords
6. Delete Record
7. Save & Exit
Enter choice (1-7): 5
Strength threshold limit (e.g. 50): 50

--- Credentials With Strength Below 50% ---
[Note] ID: 3 | Title: Lab Server Config | Category: License | Content: KEY-9941-ABCD-7721 | Entropy: 18/100

```

Figure 3: Advanced search query by keyword ("Portal") and security audit threshold filter (< 50%).

```

=====
      PASSWORD VAULT MANAGER
=====
1. Add New Login Credential
2. Add New Secure Note
3. View All Vault Items
4. Search by Title
5. Search Weak Passwords
6. Delete Record
7. Save & Exit
Enter choice (1-7): 6

--- Current Records in Vault ---

----- ALL VAULT RECORDS -----
[Login] ID: 1 | Title: Demo Portal | User: student_admin | URL: https://portal.example.com | Strength: 80/100
[Login] ID: 2 | Title: Test Forum | User: guest_user | URL: https://forum.example.com | Strength: 50/100
[Note] ID: 3 | Title: Lab Server Config | Category: License | Content: KEY-9941-ABCD-7721 | Entropy: 18/100
-----

Enter the ID of the record you want to delete (or 0 to cancel): 2
Item ID 2 deleted successfully.

```

Figure 4: Dynamic ID lookup and verified record deletion (Item ID 2).

```
=====
                PASSWORD VAULT MANAGER
=====
1. Add New Login Credential
2. Add New Secure Note
3. View All Vault Items
4. Search by Title
5. Search Weak Passwords
6. Delete Record
7. Save & Exit
Enter choice (1-7): 7

===== VAULT AUDIT REPORT =====
Total Encrypted Records : 2
Total Login Accounts    : 1
Total Secure Notes      : 1
At-Risk Items (< 50%)   : 1
Security Health Index   : 50%
=====
Vault secured. Exiting.
```

Figure 5: Automated Vault Security Audit Report & Health Index calculation upon exit.

vault.dat																	
⚙	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded Text
00000000	02	00	00	00	00	00	00	00	01	00	00	00	01	00	00	00	
00000010	44	65	6D	6F	20	50	6F	72	74	61	6C	00	00	00	00	00	D e m o P o r t a l
00000020	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000030	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000040	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000050	69	29	A5	6A	00	00	00	00	73	74	75	64	65	6E	74	5F	i) . j . . . s t u d e n t _
00000060	61	64	6D	69	6E	00	00	00	00	00	00	00	00	00	00	00	a d m i n
00000070	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000080	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000090	00	00	00	00	00	00	00	00	50	40	73	73	30	72	64	32	 P @ s s 0 r d 2
000000A0	30	32	36	00	00	00	00	00	00	00	00	00	00	00	00	00	0 2 6
000000B0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000000C0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000000D0	00	00	00	00	00	00	00	00	68	74	74	70	73	3A	2F	2F	 h t t p s : / /
000000E0	70	6F	72	74	61	6C	2E	65	78	61	6D	70	6C	65	2E	63	p o r t a l . e x a m p l e . c
000000F0	6F	6D	00	00	00	00	00	00	00	00	00	00	00	00	00	00	o m
00000100	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000110	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000120	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000130	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000140	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000150	00	00	00	00	00	00	00	00	02	00	00	00	03	00	00	00	
00000160	4C	61	62	20	53	65	72	76	65	72	20	43	6F	6E	66	69	L a b S e r v e r C o n f i
00000170	67	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	g
00000180	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000190	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000001A0	CE	29	A5	6A	00	00	00	00	4B	45	59	2D	39	39	34	31	.) . j . . . K E Y - 9 9 4 1
000001B0	2D	41	42	43	44	2D	37	37	32	31	00	00	00	00	00	00	- A B C D - 7 7 2 1

Figure 6: Hexadecimal inspection of persistent binary database file (vault.dat) - Part 1.

```
000001C0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 . . . . .
000001D0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 . . . . .
000001E0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 . . . . .
000001F0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 . . . . .
00000200 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 . . . . .
00000210 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 . . . . .
00000220 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 . . . . .
00000230 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 . . . . .
00000240 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 . . . . .
00000250 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 . . . . .
00000260 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 . . . . .
00000270 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 . . . . .
00000280 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 . . . . .
00000290 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 . . . . .
000002A0 00 00 00 00 00 00 00 00 4C 69 63 65 6E 73 65 00 . . . . L i c e n s e .
000002B0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 . . . . .
000002C0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 . . . . .
```

Figure 7: Hexadecimal inspection of persistent binary database file (vault.dat) - Part 2.